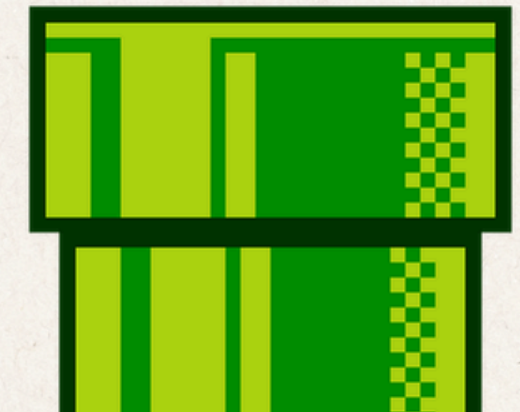
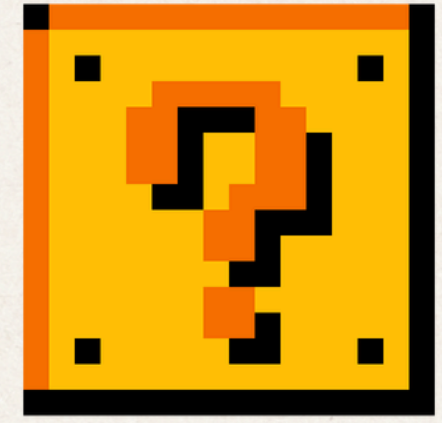




RL STUDIO



ADVISOR:
Robert Bardunias

Coltrane B

Sukhbir S

Ethan W

Gavin D-R

Vamsi S

Project Overview

- Educational tool that allows users to learn Deep Reinforcement Learning (DRL) concepts through hands-on exploration
- Built-in framework allows DLR agents to train on Atari-style games
- Training process and results are abstracted into an easy-to-use dashboard

Modular DRL Framework

- Several RL algorithms available
 - DQN
 - Rainbow
 - PPO
 - and more
- Support 104 Atari-style games
- Agents can be launched with different configurations of hyperparameters

- Tweak hyperparameters
- Launch agents

- Compare results
- Visualize performance

Interactive Dashboard

- Train agents with a simple user flow
- Results are abstracted into easily-understood metrics
- Easily tune hyperparameters and compare results
- Inspect metrics, compare runs, and visualize performance across different algorithms and games
- Didactic blurbs explaining high-level information about the algorithms

Need 🍓

- Beginners struggle to see how RL algorithms actually learn.
- Existing libraries hide key details, making it hard to understand why an agent behaves a certain way.
- Students need a hands-on framework that:
 - visualizes learning step-by-step
 - shows how hyperparameter changes affect performance
 - makes algorithms like DQN, Policy Gradient, etc. easy to explore
- Atari environments provide a familiar, standard playground for demonstrating RL concepts.

Impact 🍓

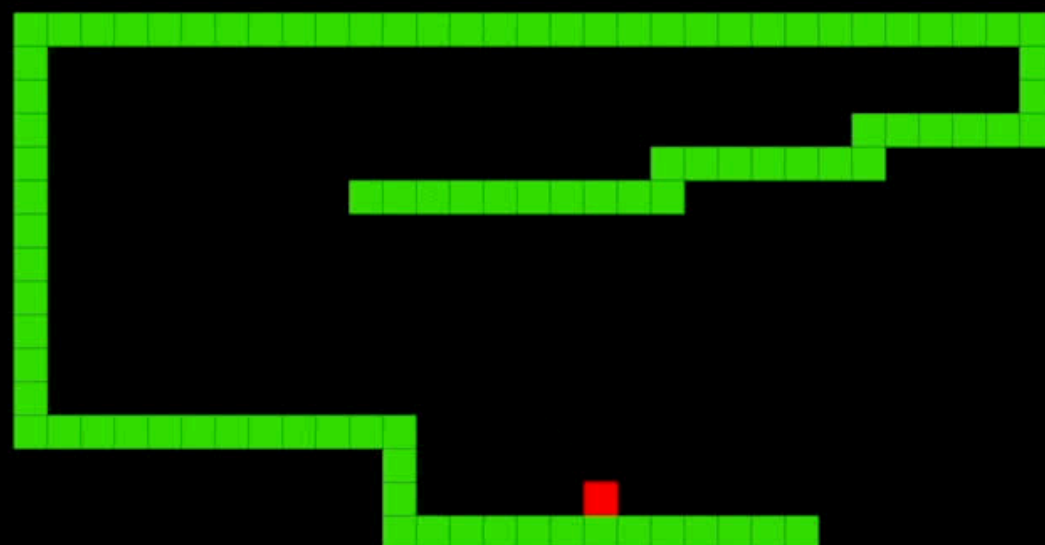
- Makes machine learning education accessible by turning abstract RL concepts into clear visuals and examples.
- Helps beginners understand the cause-and-effect between algorithm choices and outcomes.
- Gives instructors a transparent teaching tool to demonstrate RL behavior.
- Supports experimentation, curiosity, and faster learning for new ML students.



WHY THIS
PROJECT
MATTERS



TECHNOLOGY INVESTIGATION



Backend Technologies

- Stable-Baselines3 (SB3): Reliable RL baselines; easy to extend algorithms
- Gymnasium + ALE: Industry-standard Atari environments for consistent benchmarking
- PyTorch: Flexible for customizing models and debugging learning behavior
- YAML Config System: Simple, transparent hyperparameter management
- Custom Callback Library: Structured logging, JSON outputs, replay buffer access, video recording

Frontend / Visualization

- TypeScript: Safer, more maintainable code
- React: Component-based UI for dynamic RL visualizations
- Next.js: Optimized routing + server/client rendering for fast dashboards
- pnpm: Efficient package management
- Recharts: Clean, interactive charts for metrics (reward curves, loss, Q-values)

Why These Technologies

- Consistent support across 104 Atari games
- Modular, reproducible training workflows
- Clear visualization of how RL algorithms learn
- Beginner-friendly and aligned with teaching goals

BACKEND



/CONFIGS/

- YAML files define game, algorithm, and hyperparameters
- Makes experiments reproducible and easy to modify

ALGORITHMS

- This lives inside of the config file
- Applies the correct reward/penalty rules for each game depending on the algorithm & game chosen.

TRAINER CLASS

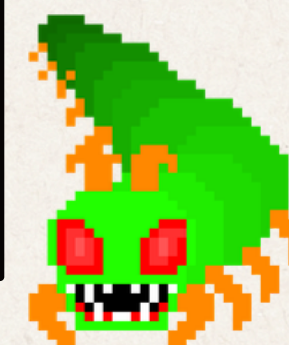
- Loads config → builds model → runs training loop
- Central controller that manages replay buffer, reward/penalty updates, callbacks, etc..

/CALLBACKS/

- JSON Logger: records rewards, losses, Q-values, epsilon
- Video Recorder: saves gameplay replays for frontend playback

OUTPUT LAYER

- Stores trained models, logs, metrics, video replays
- Powers the frontend visualization + demo videos



/Configs/



Algorithms



Trainer Class



/Callbacks/

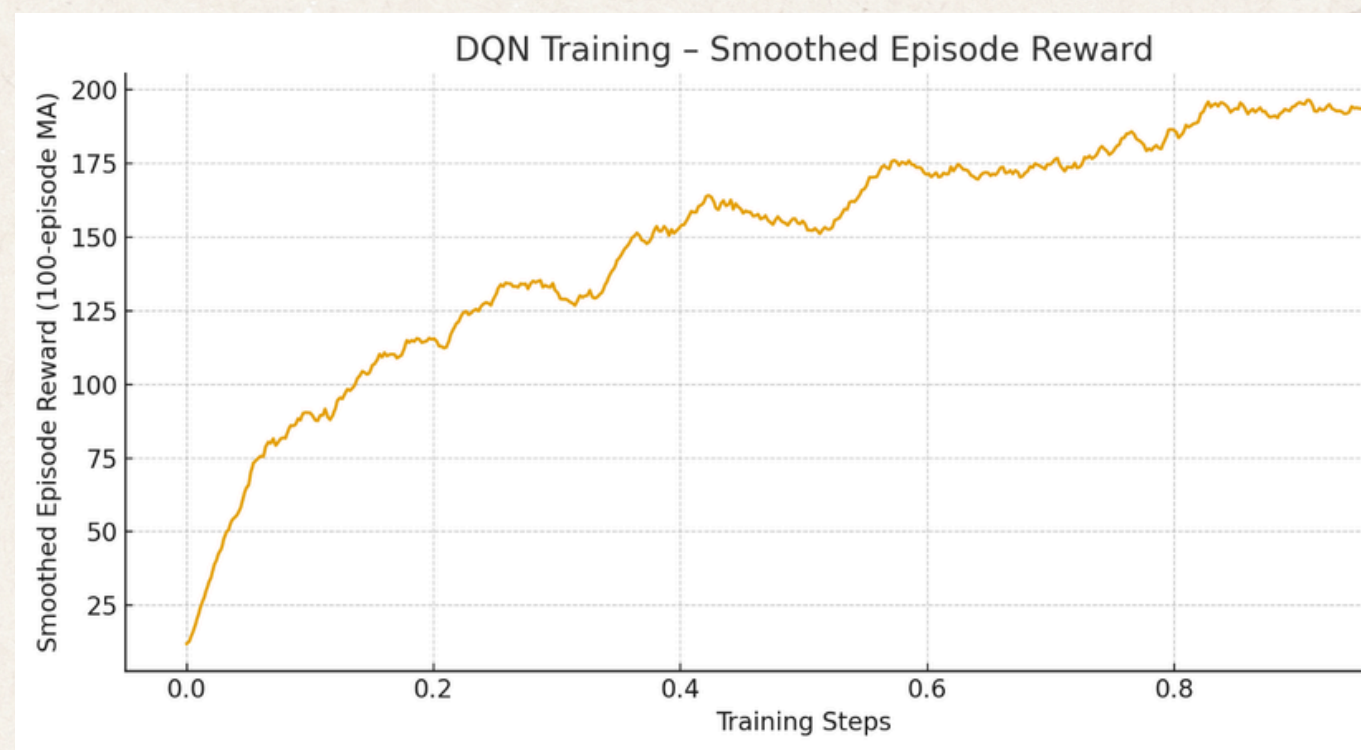
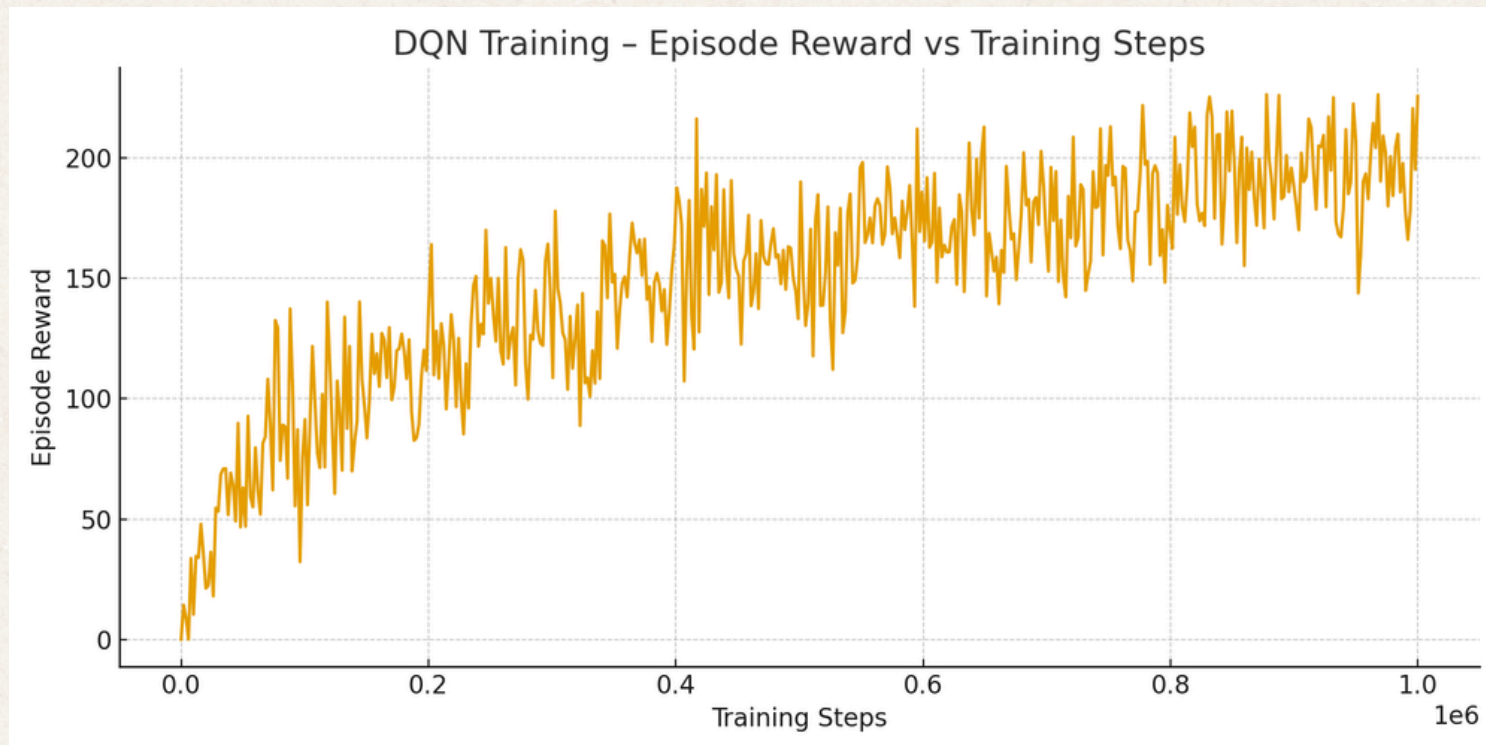


Output Layer

DATA VISUALIZATION

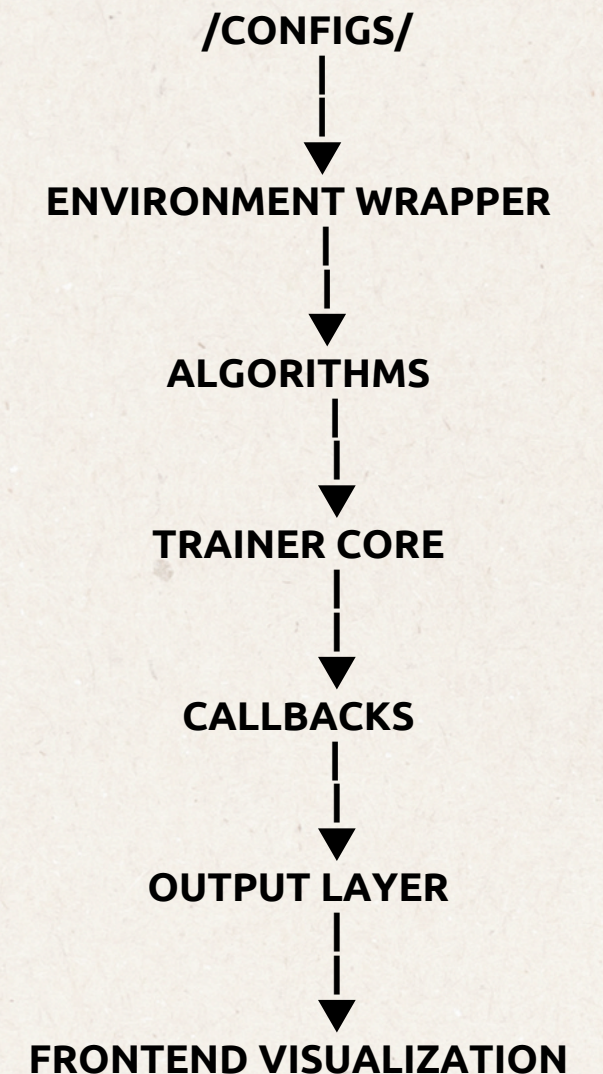


TENSORBOARD INTEGRATION



- We log all training metrics through callbacks, which write TensorBoard event files into `/runs/`.
- TensorBoard then visualizes: rewards, losses, Q-values, epsilon decay, and algorithm comparisons.
- This allows us to validate agent behavior, debug instability, and monitor convergence trends.
- The same logged data powers our frontend visualizations for consistency.

HOW THE BACKEND CONNECTS TO THE FRONTEND

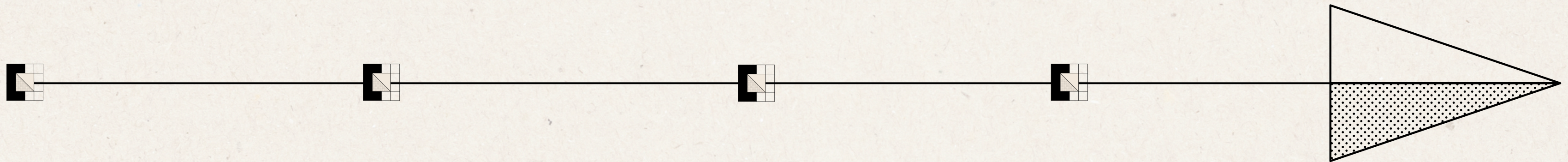


Frontend Flow



Start Pick from 104 games DQN, Rainbow, PPO Customize hyperparameters Play the selected game View the results

Work Completed



Sprint 1

Sprint 2

Sprint 3

Sprint 4

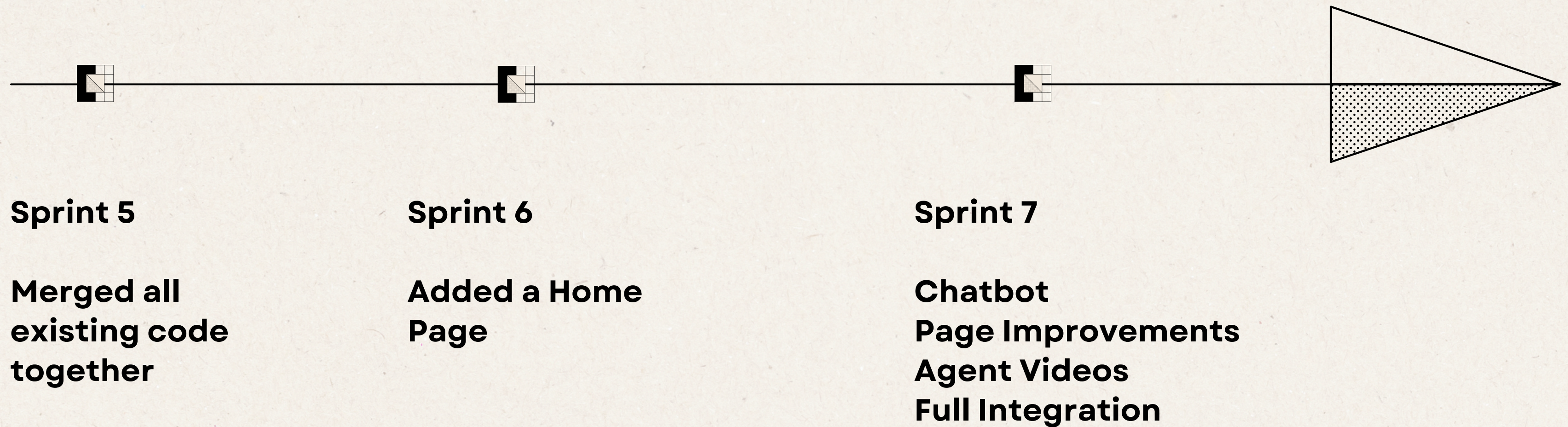
Setting up

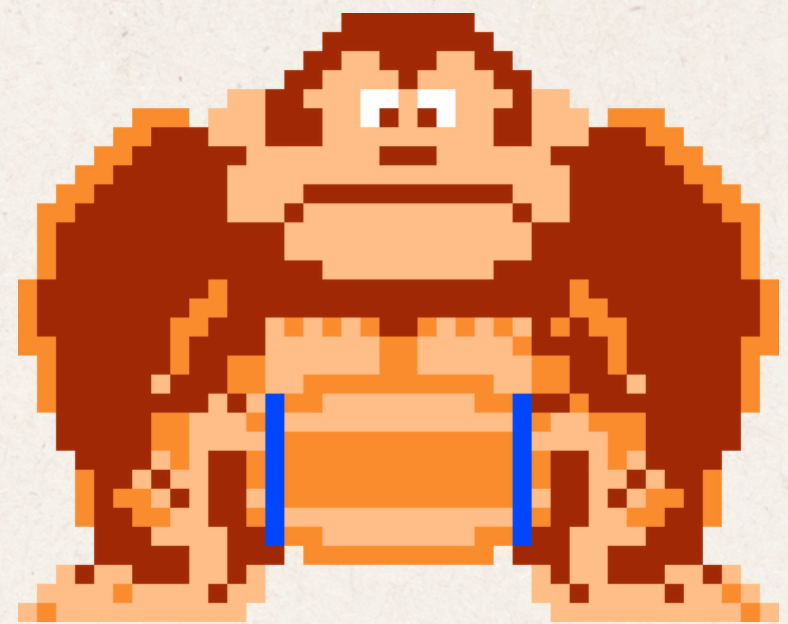
Starting work

Fleshing it out

Adding games

Work Completed





QUESTIONS

